Computer Organization

Assignment #2 - MIPS Individual Programming Project Report
Recursive Fibonacci Sequence Implementation & Mars Demonstration

學生姓名: 江源輝 **學號:** 115423067

1. C/C++ Reference Pseudocode

Below is the C++ pseudocode directly corresponding to the MIPS implementation. It defines the base cases, the double-recursion calculation, the input validation loop (0 <= n <= 20), and the comma-formatted sequence output.

```
#include <iostream>
using namespace std;

// Recursive procedure matching the MIPS fib implementation
int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n;
    // Input validation loop: 0 <= n <= 20 with ask-again mechanism
    while (true) {
        cout << "Enter n (0-20): ";
        cin >> n;
        if (n >= 0 && n <= 20) {
            break;
        }
        cout << "Invalid input. Please enter a value from 0 to 20.\n";
    }

    // Sequence output loop
    cout << "Fibonacci sequence: ";
    for (int i = 0; i <= n; i++) {
        cout << fib(i);
        if (i < n) {
            cout << ", ";
        }
    }
    cout << "\n";

    return 0;
}
```

2. MIPS Assembly Source Code

The assembly program below was written following the project requirements and verified in the MARS simulator. It strictly avoids loops in fib, properly utilizes the 12-byte stack frame, and adheres to standard MIPS calling conventions.

```
       .data
prompt:.asciiz "Enter n (0-20): "
error_msg:.asciiz "Invalid input. Please enter a value from 0 to 20.\n"
result_msg:.asciiz "Fibonacci sequence: "
comma:.asciiz ", "
newline:.asciiz "\n"

       .text
       .globl main

# main function

main:
# 1. input valid loop (0<=n<=20)
input_loop:
       la $a0, prompt #print input prompt
       li $v0, 4
       syscall

       li $v0, 5 #read int n from user
       syscall
       move $s1, $v0 #save n in $s1(callee-saved)

       bltz $s1, input_error #if n<0 branch to error
       li $t0, 20
       slt $at, $t0, $s1
       bne $at, $zero, input_error #if n>20 branch to error
       j start_sequence #input is valid proceed to sequence generation
input_error:
       la $a0, error_msg # print error message
       li $v0, 4
       syscall # prompt again
       j    input_loop
# 2. sequence output loop(for i=0 to n)
start_sequence:
       la $a0, result_msg #print "Fibonacci sequence: "
       li $v0, 4
       syscall

       li $s0, 0 #loop counter i =0   (saved in $s0)
print_loop:
       slt $at, $s1, $s0
       bne $at, $zero, print_done # if i>n,exit loop

       move $a0, $s0 #pass argument i in $a0
       jal fib #call recursive procedure fib(i)

       move $a0, $v0 #print returned fibonacci number f(i)
       li $v0, 1
       syscall
```

```
       beq $s0, $s1, skip_comma# skip comma for the last term(when i==n)

       la $a0, comma #print ", "
       li $v0, 4
       syscall
skip_comma:
       addi $s0, $s0, 1 #i++
       j print_loop #repeat loop
print_done:
       la $a0, newline #print newline "\n"
       li $v0, 4
       syscall

       li $v0, 10 # exit program cleanly(syscall 10)
       syscall

#recursive procedures: fib
#arguments: $a0 =n
#returns: $v0=f(n)
#stack frame size: 12 BYTES($ra,$a0,$s0)


fib:
    # 1. Prologue: Allocate stack space and save registers
       addi $sp, $sp, -12
       sw $ra, 8($sp) #save return address
       sw $a0, 4($sp) #save current argument n
       sw $s0, 0($sp) #save $s0 to preserve outer value
    # 2. Base Case Check: if n < 2
       slti $t0, $a0, 2
       bne $t0, $zero, fib_base
    # 3. Recursive Call 1: fib(n - 1)
       addi $a0, $a0, -1 # $a0=n-1
       jal fib # Recursive call
       move $s0, $v0 # Preserve result of fib(n - 1) in $s0
    # 4. Recursive Call 2: fib(n - 2)
       lw $a0, 4($sp) # Restore original n from stack
       addi $a0, $a0, -2  # $a0 = n - 2
       jal fib # Recursive call, result in $v0
    # 5. Combine Results: F(n) = fib(n - 1) + fib(n - 2)
       add $v0, $s0, $v0 # $v0 = $s0 + $v0
       j fib_exit
fib_base:
    # Base Case: F(0) = 0, F(1) = 1 (return n directly)
       move $v0, $a0
fib_exit:
    # 6. Epilogue: Restore registers and deallocate stack space
       lw $s0, 0($sp) # Restore $s0
       lw $a0, 4($sp) # Restore $a0
       lw $ra, 8($sp) # Restore return address $ra
       addi $sp, $sp, 12 # Deallocate 12 bytes
       jr $ra # Return to caller
```

3. Register Usage & Labeled Stack Frame Layout

3.1 Register Usage Table

Register	Purpose in Program	Calling Convention Class
`$s1`	Main: stores user input n across function calls and loop iterations	Callee-saved (preserved across calls)
`$s0`	Main: loop counter i (0 to n). Inside fib: preserves F(n-1) across calls	Callee-saved (preserved across calls)
`$a0`	Argument register: passes n into fib(n), and string/integer to syscalls	Argument register (caller-saved)
`$v0`	Return value of fib(n), and syscall service request code	Return value register (caller-saved)
`$ra`	Return address register, holds target address to return via jr $ra	Return address register
`$sp`	Stack pointer pointing to the current active top of the runtime stack	Stack pointer

3.2 Labeled Stack Frame Layout (12 Bytes)

Each activation of procedure fib dynamically allocates exactly 12 bytes on the stack (addi $sp, $sp, -12). The layout and byte displacements are explicitly illustrated below:

```
High Address  +-----------------------------------------+
              |  $ra  : Return address to caller (8 bytes) |  8($sp)
              +-----------------------------------------+
              |  $a0  : Initial argument n (4 bytes)       |  4($sp)
              +-----------------------------------------+
Low Address   |  $s0  : Saved $s0 from parent (0 bytes)    |  0($sp)
              +-----------------------------------------+  <-- Current $sp
```

Detailed Stack Field Mechanics:
1. Return Address at 8($sp): Because fib executes nested jal instructions, the hardware automatically overwrites $ra. Saving $ra at 8($sp) guarantees that after all recursive subtrees resolve, the program can restore $ra and jump back to the exact return instruction via jr $ra.
2. Initial Argument n at 4($sp): Evaluating the first branch requires setting $a0 = n - 1. Storing the original n at 4($sp) allows the procedure to reload the initial argument (lw $a0, 4($sp)) and safely compute n - 2 for the second recursive branch.
3. Callee-Saved Register at 0($sp): The result of the first recursive call F(n-1) returns in $v0. To prevent it from being overwritten during the evaluation of the second branch F(n-2), it is preserved in $s0. According to MIPS calling conventions, the parent frame's $s0 must be preserved upon entry and restored upon exit.

4. Test Inputs and Verification Results

The program was thoroughly tested under both invalid edge cases and normal sequence generations in the MARS simulator. The outputs match all assignment requirements:

4.1 Invalid Input Validation (Boundary Test)

Test Case: Entered 50 (or 21, outside the range 0 <= n <= 20). The program correctly caught the error, displayed "Invalid input. Please enter a value from 0 to 20.", and asked again.

```
Enter n (0-20): 50
Invalid input. Please enter a value from 0 to 20.
Enter n (0-20): 5
Fibonacci sequence: 0, 1, 1, 2, 3, 5
```

4.2 Valid Test Case: n = 0

Expected output: 0. Verified output: "Fibonacci sequence: 0".

```
Enter n (0-20): 0
Fibonacci sequence: 0

-- program is finished running --
```

4.3 Valid Test Case: n = 1

Expected output: 0, 1. Verified output: "Fibonacci sequence: 0, 1".

```
Enter n (0-20): 1
Fibonacci sequence: 0, 1

-- program is finished running --
```

4.4 Valid Test Case: n = 5

Expected output: 0, 1, 1, 2, 3, 5. Verified output: "Fibonacci sequence: 0, 1, 1, 2, 3, 5".

```
Enter n (0-20): 5
Fibonacci sequence: 0, 1, 1, 2, 3, 5

-- program is finished running --
```

4.5 Valid Test Case: n = 10

Expected output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55. Verified output matches.

```
Enter n (0-20): 10
Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

-- program is finished running --
```

5. Disclosure Use of AI Tools

```
I developed and implemented the initial C++ reference logic independently.
I utilized Google Antigravity (Gemini) as an interactive pedagogical tool to:
1. Guide me step-by-step through translating my C++ logic into MIPS assembly language.
2. Design and verify the 12-byte stack frame layout and MIPS calling conventions.
3. Review and ensure completeness of the technical explanations regarding program flow and
register interactions.
```

> All submitted code, stack frame mechanics, and execution flows have been thoroughly verified, tested, and fully understood by myself.